Recording Changes to the Repository

Tuesday, November 25, 2025 10:37 PM

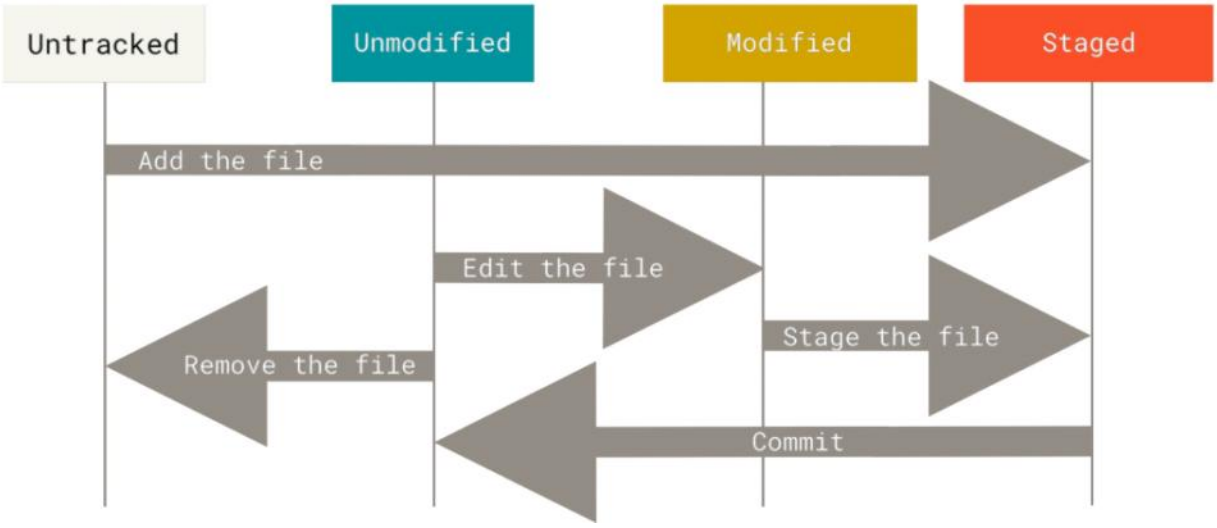
<u>Remember</u> that each **file** in your **working directory** can be in one of **two states**

1. <u>Tracked</u>

   Files that were in the **last snapshot**, as well as any **newly staged** files. These files can be **unmodified**, **modified**, or **staged**

2. <u>Untracked</u>

   Files in your working directory that were **not** in your **last snapshot** and are **not** in your **staging area**



<u># Checking the Status of Files</u>

1. Go to project directory

   cd /path/to/directory

2. Use the following command to see the status of files

   git status

   Sample output of this command in a clean directory

On branch main

No commits yet

nothing to commit (create/copy files and use "git add" to track)

3. Let's create a new file named README.md
   a. *echo 'Temporary DevOps Project' > README.md*
   b. *git status*

```
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        README.md

nothing added to commit but untracked files present (use "git add" to track)
```

Means that Git sees a file you didn't have in the previous snapshot (commit), and which hasn't yet been staged

Tracking New Files

1. Go to project directory

   cd /path/to/directory

2. Use the following command to see if any new file is available

   git status

3. Use below command to start tracking new files

   git add <filename>

4. Now run following status command

   git status

```
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   README.md
```

Means that Git sees a file you didn't have in the previous snapshot (commit), and which hasn't yet been staged

Staging Modified Files

1. Go to project directory

   cd /path/to/directory

2. Let's change a file that was already tracked

3. Now run following status command

   git status

```
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")
```

4. Now add this modified file and check status

   git add README.md

   git status

```
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   README.md
```

5. Suppose you remember another change and want to update file before committing

   vim README.md

   git status

```
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   README.md

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md
```

Which file version will be committed now if you run following command?

git commit

6. If you **modify** a file **after** you **run git add**, you have to run **git add again** to **stage** the **latest version** of the file

   git add README.md

Ignoring Files

Often, you'll have a **class** of **files** that you **don't want** Git to automatically **add or even show** you as being **untracked**

a. These are generally **automatically** generated **files** such as **log files** or
b. Files **produced** by your **build system**.

In such cases, you can create a file **listing patterns** to match them named **.gitignore**.

Setting up a **.gitignore** file for your **new repository** is generally a good idea so you don't a**ccidentally commit** files that you really **don't want** in your Git repository. Here is an example .gitignore file

```
$ cat .gitignore
*.[oa]
*~
```

The *first line* tells Git to **ignore** any **files** ending in ".o" or ".a"

The *second line* tells Git to ignore all files whose names end with a tilde (~)

Rules for the patterns

1. Blank lines or lines starting with # are ignored
2. Standard glob patterns work, and will be applied recursively throughout the entire working tree. Means simplified regular expressions

   a. ***** matches zero or more characters
   b. **[abc]** matches any character inside the brackets (in this case a, b, or c)
   c. **(?)** matches a single character

d. **([0- 9])** matches any character between them (in this case 0 through 9)
e. **a/**/z** would match a/z, a/b/z, a/b/c/z, and so on

3. You can start patterns with a forward slash (/)
4. You can end patterns with a forward slash (/) to specify a directory.
5. You can negate a pattern by starting it with an exclamation point (!).

```
# ignore all .a files
*.a

# but do track lib.a, even though you're ignoring .a files above
!lib.a

# only ignore the TODO file in the current directory, not subdir/TODO
/TODO

# ignore all files in any directory named build
build/

# ignore doc/notes.txt, but not doc/server/arch.txt
doc/*.txt

# ignore all .pdf files in the doc/ directory and any of its subdirectories
doc/**/*.pdf
```

For more details please visit this link

Viewing your Staged and Unstaged Changes

Let's ask following questions from ourself

1. What have you changed but not yet staged?
2. What have you staged that you are about to commit?

Are these questions answered by *git status* command ? Yes, this commands addresses these questions but very generally.

Following command will show you exact lines added or removed

git diff

```
diff --git a/README.md b/README.md
index b112b41..c4ce5d3 100644
--- a/README.md
+++ b/README.md
@@ -1,2 +1,3 @@
 #Comment in a readme file
+#Another comment in a readme file
 Temporary DevOps Project
```

Following command will show you exact lines added or removed between **staged** and **committ**

git diff --staged

```
$ git diff --staged
diff --git a/README.md b/README.md
index 905234a..b112b41 100644
--- a/README.md
+++ b/README.md
@@ -1 +1,2 @@
+#Comment in a readme file
 Temporary DevOps Project
```

Committing your Changes

The simplest way to commit is to type git commit

git commit

```
$ git commit -m '.gitignore file added'
[main ec0db1f] .gitignore file added
 2 files changed, 4 insertions(+)
 create mode 100644 .gitignore
```

Removing Files

The simplest way to remove file

git rm <filename>

```
$ git rm test2
rm 'test2'
```

```
$ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        deleted:    test2
```

Moving Files

The simplest way to remove file

git mv <filename_from> <filename_to>